

# Collaborating with Git

# Content

- GitFlow
  - A real-life example
- GitLab Issues
- Merge Requests
- Code Reviews

# GitFlow

# What is GitFlow?

- GitFlow is a branching model designed to structure and streamline software development
- It defines a clear workflow for managing code versions, ensuring stability, and facilitating collaboration in a team
- Introduced by Vincent Driessen (2010), it is widely used in Agile software development

# What is GitFlow?

- GitFlow is a model, not a strict rulebook
- The core concepts remain the same, but companies adapt it
- Common Variations
  - Classic GitFlow (with *release* branches)
  - Simplified GitFlow (no *release* branches)
  - Trunk-Based Development (no *dev*, everything on *main*)
- The best GitFlow depends on your team's workflow and deployment strategy

# Why should we use GitFlow?

- Separates stable code from ongoing development, reducing risks
- Allows multiple features to be developed in parallel on isolated branches, increasing efficiency
- Ensures quality and stability by integrating testing before release
- Facilitates collaboration using Merge Requests for code review
- Keeps track of releases using tags and structured versioning

# When should we use GitFlow?

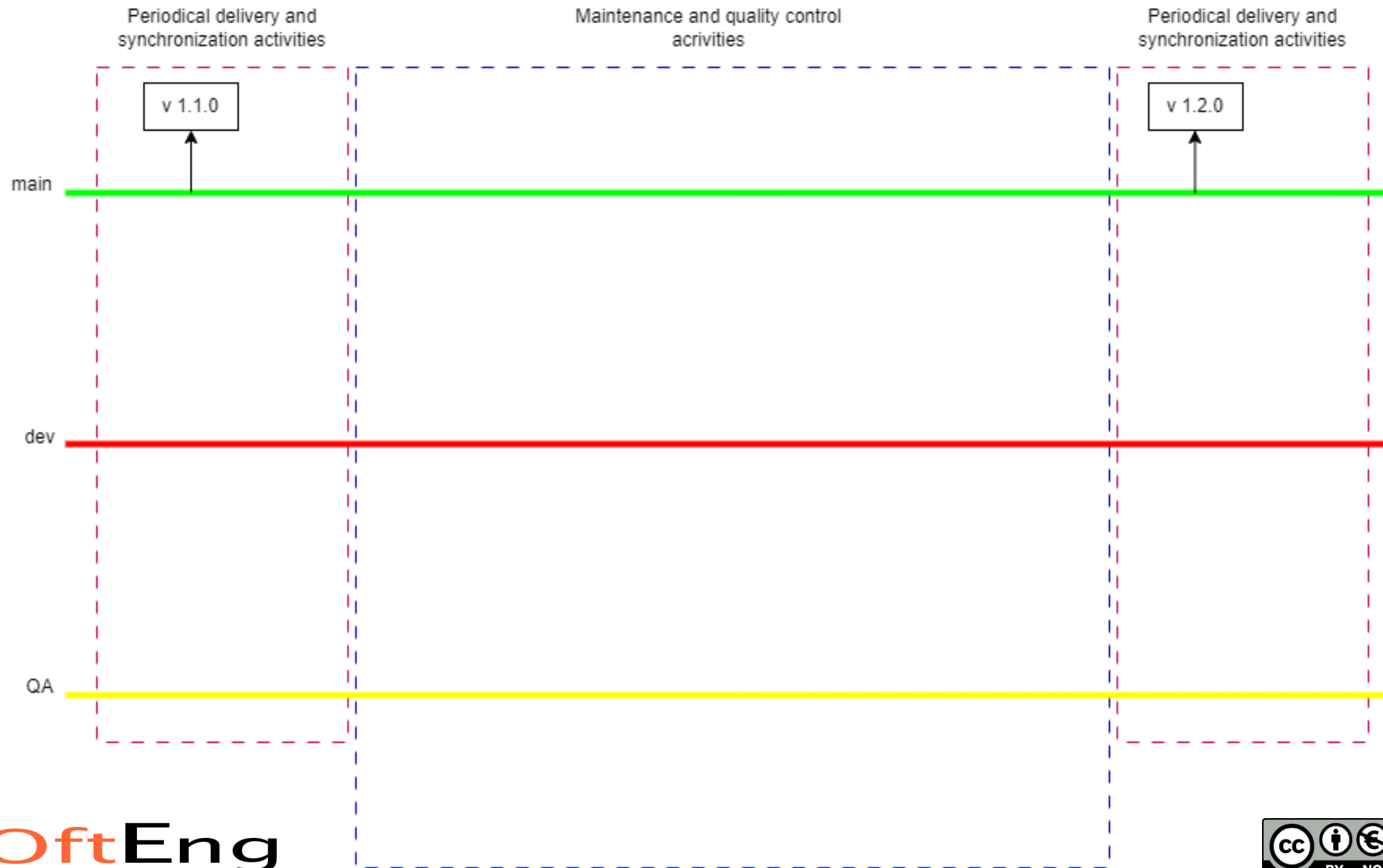
- ALWAYS! (or at least in structured projects)
- Especially beneficial for projects with periodic releases (e.g., Agile methodologies)
- Essential for teams working in CI/CD environments, where automated builds and deployments rely on a structured branching model

# GitFlow – A real-life example



# Goal

- Periodic delivery of a new product version, ensuring predictable releases
- Maintaining high-quality standards, with a structured Quality Assurance process
- Enabling parallel work between development and Quality Assurance, avoiding bottlenecks
- Isolating production code from development and testing, allowing
  - Safe production bug fixes without interfering with ongoing development
  - A stable environment for Quality Assurance to validate before releasing new versions



# Branch structure – persistent branches

- These branches must remain stable and serve as reference points for the production process.
- *main* branch
  - Production-ready code
- *dev* branch
  - Base for new features
- *QA* branch
  - Testing and integration branch

# Persistent branches – *main* branch

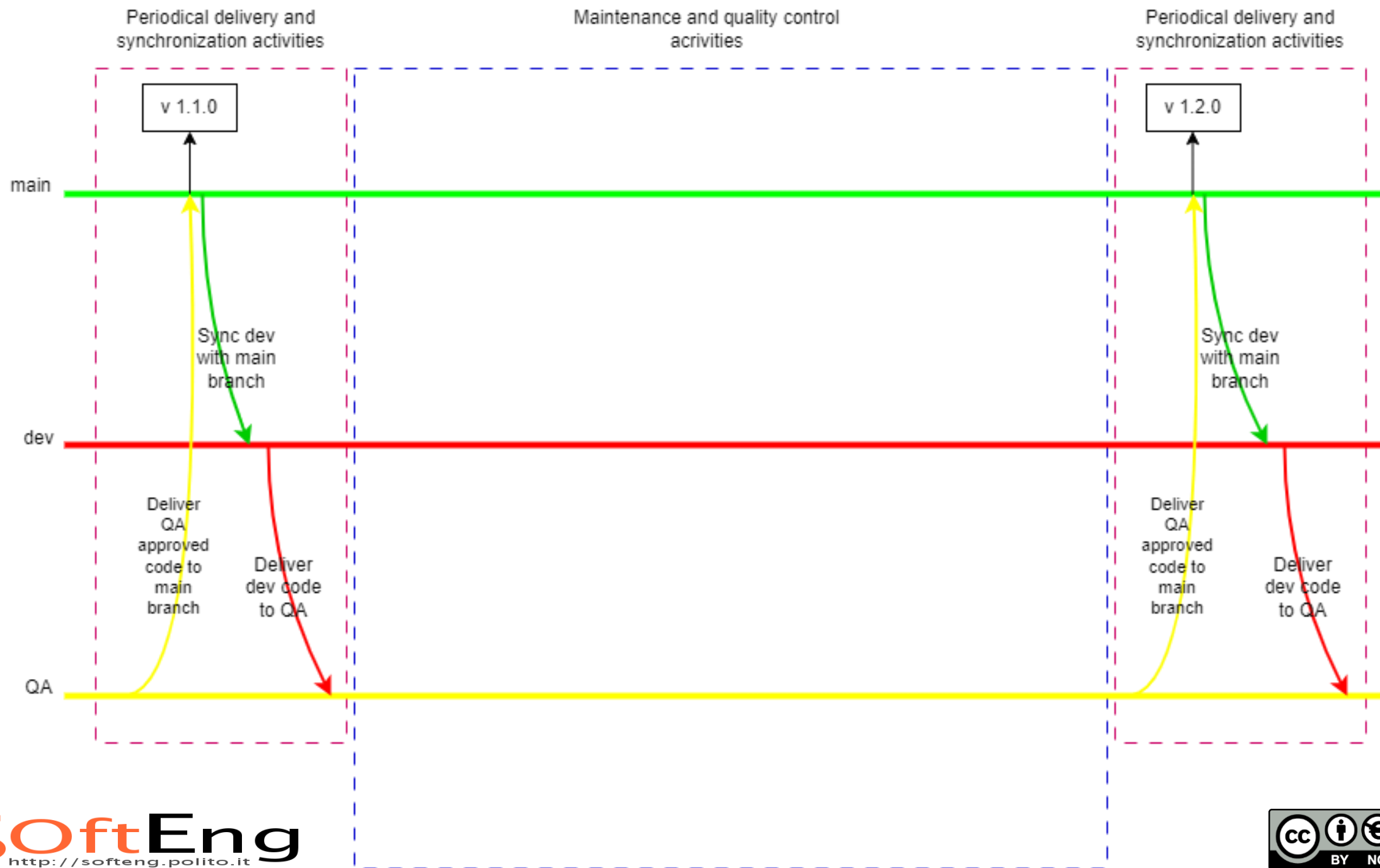
- Contains the latest stable and production-ready code
- Only approved, tested code is merged here
- Releases are marked using tags, such as v1.0.0
- Automated CI/CD pipelines trigger builds and deployments when code is merged
- Fixes for production bugs are handled using *hotfix*/ branches

# Persistent branches– *dev* branch

- The main working branch where new features are integrated before testing
- Developers create *feature/* branches from *dev*
- *feature/* branches are merged into *dev* after development and unit testing
- Periodically, *dev* is merged into *QA* for integration testing

# Persistent branches– QA branch

- QA branch is used for quality assurance and integration testing
- The Quality Assurance team performs
  - Regression testing to ensure existing features work
  - Integration testing to verify new code does not break the system
- If a bug is found, a *bugfix*/ branch is created to address it
- Once testing is complete, QA is merged into *main*, and a new release tag is created



# Branch structure – ephemeral branches

- We avoid commits on persistent branches (*main, dev, QA*)
- The use of ephemeral branches isolates every single development in its branch avoiding any impact on other branches
- The use of ephemeral branches allows to create merge requests
  - Enable code reviews before merging
  - Maintain clean and structured code
  - Ensure that only validated changes reach persistent branches
- Three types of ephemeral branches
  - *feature/*
  - *hotfix/*
  - *bugfix/*



# Ephemeral branches – *feature* branches

- Develop new functionalities before merging into *dev*
- Branch Naming Convention
  - *feature/#issueID-featureDescription*
  - *issueID* is the GitLab/Jira/GitHub... ID of the issue associated to the new feature
- Workflow
  - Branch is created from *dev*
  - Code is developed, including unit/integration tests
  - Merge Request is opened for team review and approval
  - Once approved, the branch is merged back into *dev* and deleted

# Ephemeral branches – *hotfix* branches

- Fix critical bugs found in production (*main*)
- Branch Naming Convention
  - *hotfix/#issueID-bugDescription*
  - *issueID* is the GitLab/Jira/GitHub... ID of the issue associated to the bug
- Workflow
  - Branch is created from *main*
  - Bug is fixed and tested
  - Merge Request is opened for team review and approval
  - Once approved, the branch is merged back into *main* and deleted
  - A new tag and release is created on *main* branch
  - If necessary, the fix is cherry-picked into *dev* and *QA*

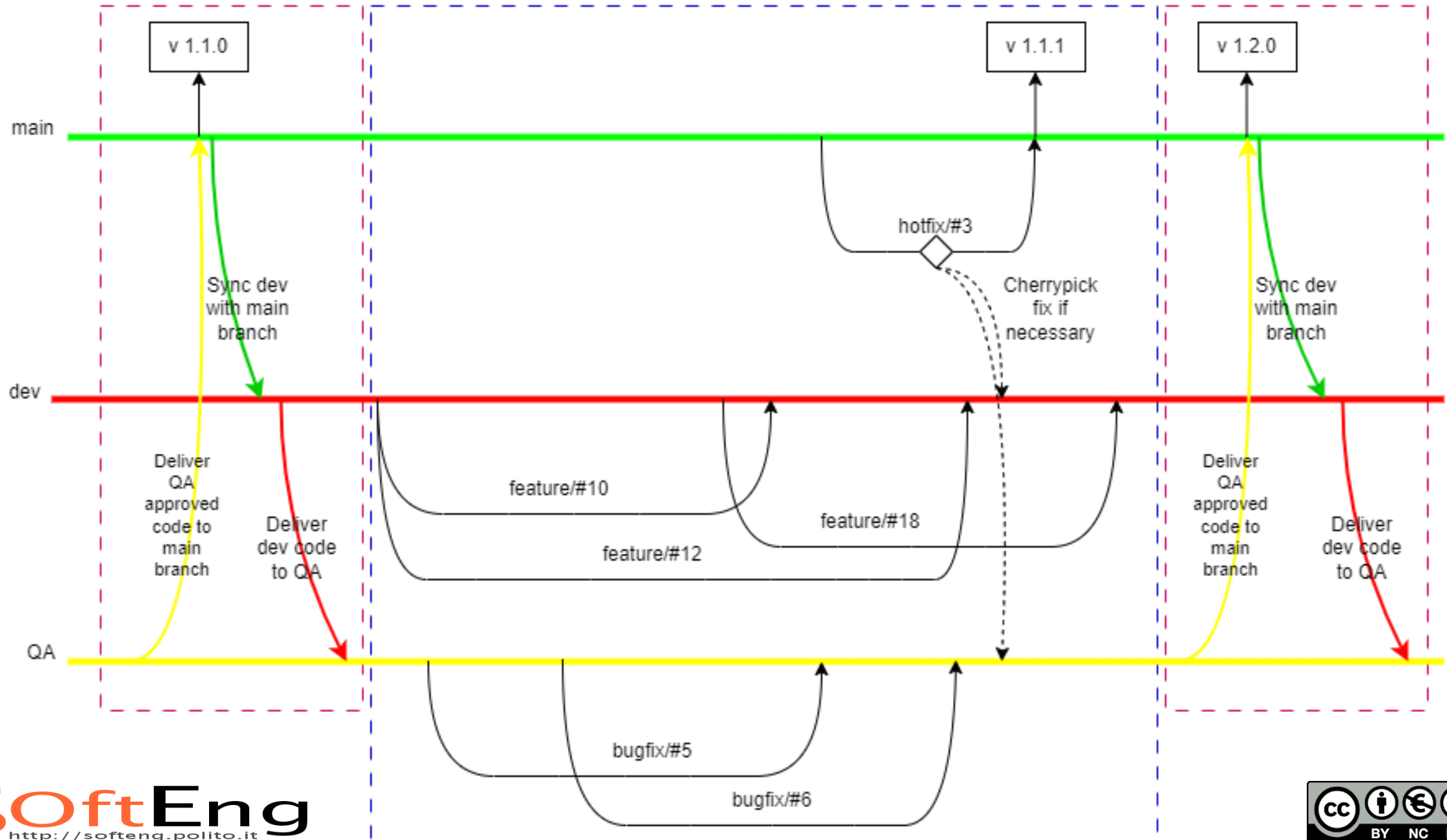
# Ephemeral branches – *bugfix* branches

- Fix bugs found during testing (QA branch)
- Branch Naming Convention
  - *bugfix/#issueID-bugDescription*
  - *issueID* is the GitLab/Jira/GitHub... ID of the issue associated to the bug
- Workflow
  - Branch is created from QA when Quality Assurance team finds a bug
  - Bug is resolved and tested
  - Merge Request is opened for team review and approval
  - Once approved, the branch is merged back into QA and deleted
  - If necessary, it is also cherrypicked to *dev*

Periodical delivery and  
synchronization activities

Maintenance and quality control  
activities

Periodical delivery and  
synchronization activities



# GitFlow – best practices

- Keep *main* stable
  - Only merge QA-approved code
- Use meaningful Merge Requests
  - Provide clear descriptions and references, add images if GUI is impacted
- Write unit tests before merging to *dev*
- Keep *dev* in sync
  - Merge from *main* after releases
- Follow a strict naming convention for branches
- Use CI/CD automation
  - Trigger builds and tests on merges to main

# GitFlow – bad practices to avoid

- Working directly on persistent branches
  - ❌ Modifying *main*, *dev*, or *QA* without a separate ephemeral *hotfix/*, *feature/* or *bugfix/* branch
  - ✅ Always use ephemeral branches and create Merge Requests
- Not syncing branches regularly
  - ❌ Leaving *dev* outdated after a production bug fix
  - ✅ Regularly merge *main* into *dev* after releases
- Not using meaningful branch names
  - ❌ Using *feature/experiment1* instead of *feature/1234-add-login-form*
  - ✅ Stick to the naming conventions (*feature/#issueld-featureDescription*)

# GitFlow – bad practices to avoid

- Forgetting to delete merged branches
  - ✗ Leaving old feature branches that clutter the repo
  - ✓ Always delete ephemeral branches after merging
- Creating personal branches in the shared repo
  - ✗ Using *johns-branch*, *my-test*, *fix-quick*, or similar personal names
  - ✓ Branches in the shared repository must follow team conventions and be accessible to all
  - ✓ If personal work is needed, use a personal fork and keep it synced

# GitFlow – Key takeaways

- GitFlow provides structure for (especially Agile) development with well-defined branch roles
- Persistent branches (*main*, *dev*, *QA*) ensure stability and separation of concerns
- Ephemeral branches (*feature/*, *hotfix/*, *bugfix/*) keep work isolated and reviewable
- Merge Requests are essential for code reviews and quality control
- Synchronization between branches ensures all fixes and features reach production



# Gitlab issues

# What's a GitLab issue

- GitLab Issues are a simple way to track tasks, bugs, and new features. They help teams organize work, assign responsibilities, and document decisions.
- Creating an Issue allows developers to describe what needs to be done before writing code. Issues can be linked to branches and Merge Requests, making collaboration clear and traceable.

# Using GitLab issues in this course

- In the context of the course project, GitLab Issues can be used in two ways
- Within the team, in your group repository ([git-softeng.polito.it/se20xx-xx/group-xx/projectName](https://git-softeng.polito.it/se20xx-xx/group-xx/projectName))
  - To plan tasks, track bugs, and organize development
  - To assign work and communicate clearly with teammates
- Towards the teaching staff, in the general repository ([git-softeng.polito.it/se20xx-xx/projectName](https://git-softeng.polito.it/se20xx-xx/projectName))
  - To report inconsistencies in the API specifications
  - To report problems or bugs in the provided code
- Using Issues helps keep communication organized, documented and traceable

# Creating a GitLab issue

- Go in the “Plan” section in the repository
- Choose the "Issues" subsection in the “Plan” tab
- Click "New Issue"
- Add a clear title and descriptive explanation

# Creating a GitLab issue

Stefano Mancini / Lab0 Simple Calculator / Issues / New

Project

Lab0 Simple Calculator

Pinned

Issues

Merge requests

Manage

Plan

Issues

Issue boards

Milestones

Wiki

Code

Build

Secure

Deploy

Help

Admin Area

New Issue

Title (required)

Power calculation

Type

Issue

Description

Preview

**B** *I* U | **≡** `</>` 🔗 **≡** **≡** **≡**

Add support for power calculation (exponentiation) in the calculator module.  
This includes computing expressions like a^b, where a is the base and b is the exponent.

Switch to rich text editing

Add [description templates](#) to help your contributors to communicate effectively!

☐ This issue is confidential and should only be visible to team members with at least Reporter access.

Assignee

Unassigned

Assign to me

Due date

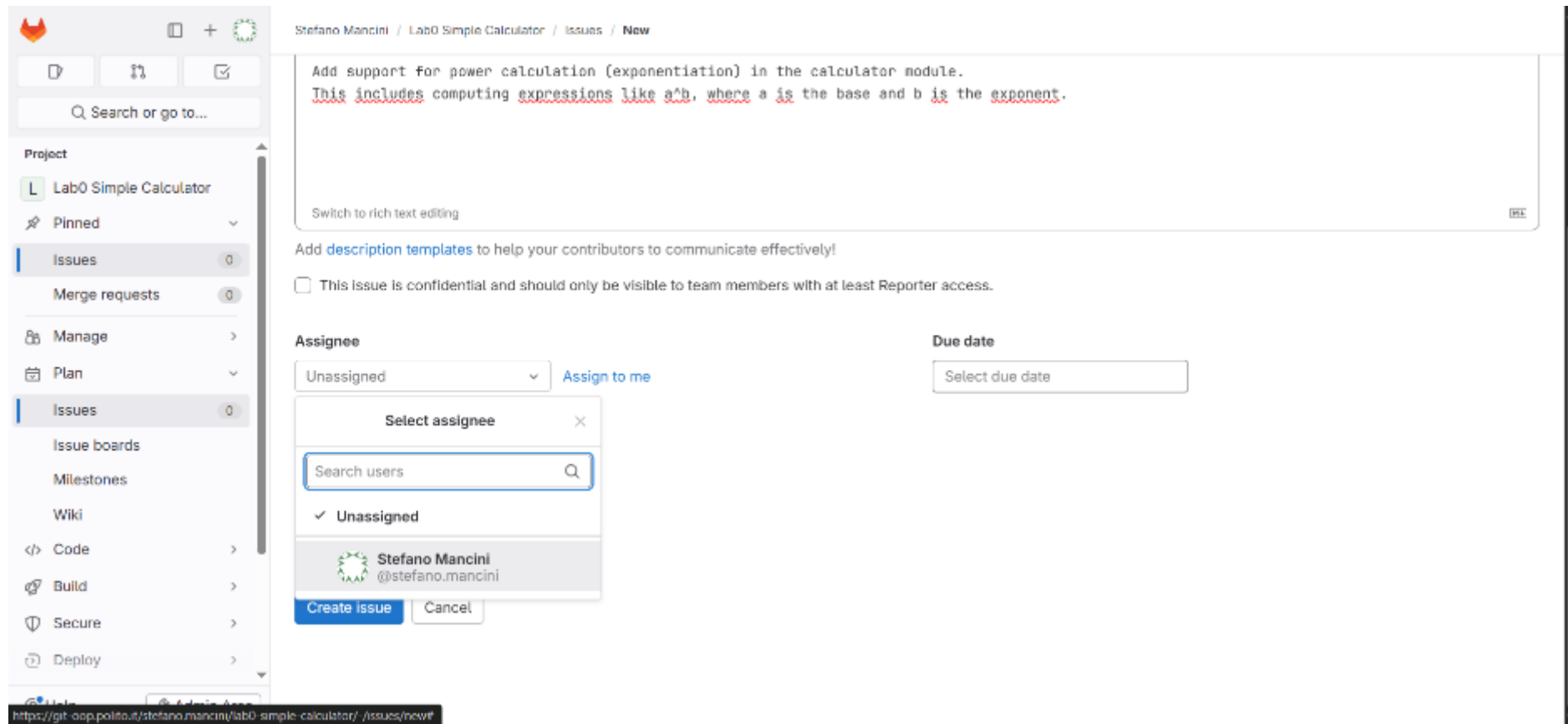
Select due date

Milestone

# GitLab issue configuration

- (Optional) Assign it to yourself or a teammate
- (Optional) Add labels to categorize the task
- (Optional) Customize the issue with the options offered by GitLab
  - Due date
  - Milestone
  - ...

# GitLab issue configuration



# GitLab issue features

- GitLab allows to create a branch and a Merge Request directly from the issue
  - Branch is named correctly and clearly
  - MR is automatically opened from the new branch, user chooses the destination branch
- The issue, branch, and MR are automatically connected
- When the MR is merged, the issue can be closed automatically



# GitLab issue features

The screenshot displays the GitLab interface for an issue titled "Power calculation" in the "Lab0 Simple Calculator" project. The issue is created by Stefano Mancini and is currently open. The description states: "Add support for power calculation (exponentiation) in the calculator module. This includes computing expressions like  $a^b$ , where  $a$  is the base and  $b$  is the exponent." The issue has 0 votes and 0 comments. A sidebar on the left contains navigation links for the project, including "Issues", "Merge requests", "Manage", "Plan", "Issue boards", "Milestones", "Wiki", "Code", "Build", "Secure", and "Deploy". A dropdown menu is open, showing options to "Create merge request and branch" and "Create branch". The "Create merge request and branch" option is selected, and the "Target project" is set to "Stefano Mancini / Lab0 Simple C..". The "Branch name" is "1-power-calculation" and the "Source (branch or tag)" is "main". The "Create merge request" button is visible at the bottom of the dropdown. The right sidebar shows the "Assignee" as Stefano Mancini, "Labels" as None, "Milestone" as None, "Weight" as "This feature is locked. Learn more", "Due date" as None, "Time tracking" as "No estimate or time spent", "Confidentiality" as "Not confidential", and "1 Participant".

# Merge requests

# What's a Merge Request

- A Merge Request (MR) is a GitLab feature (GitHub calls them Pull Request) that allows developers to propose changes made in one branch to be merged into another.
- It is a collaborative tool that lets the team review the code before it becomes part of the main project, discuss changes and suggest improvements and track what has been modified and why.

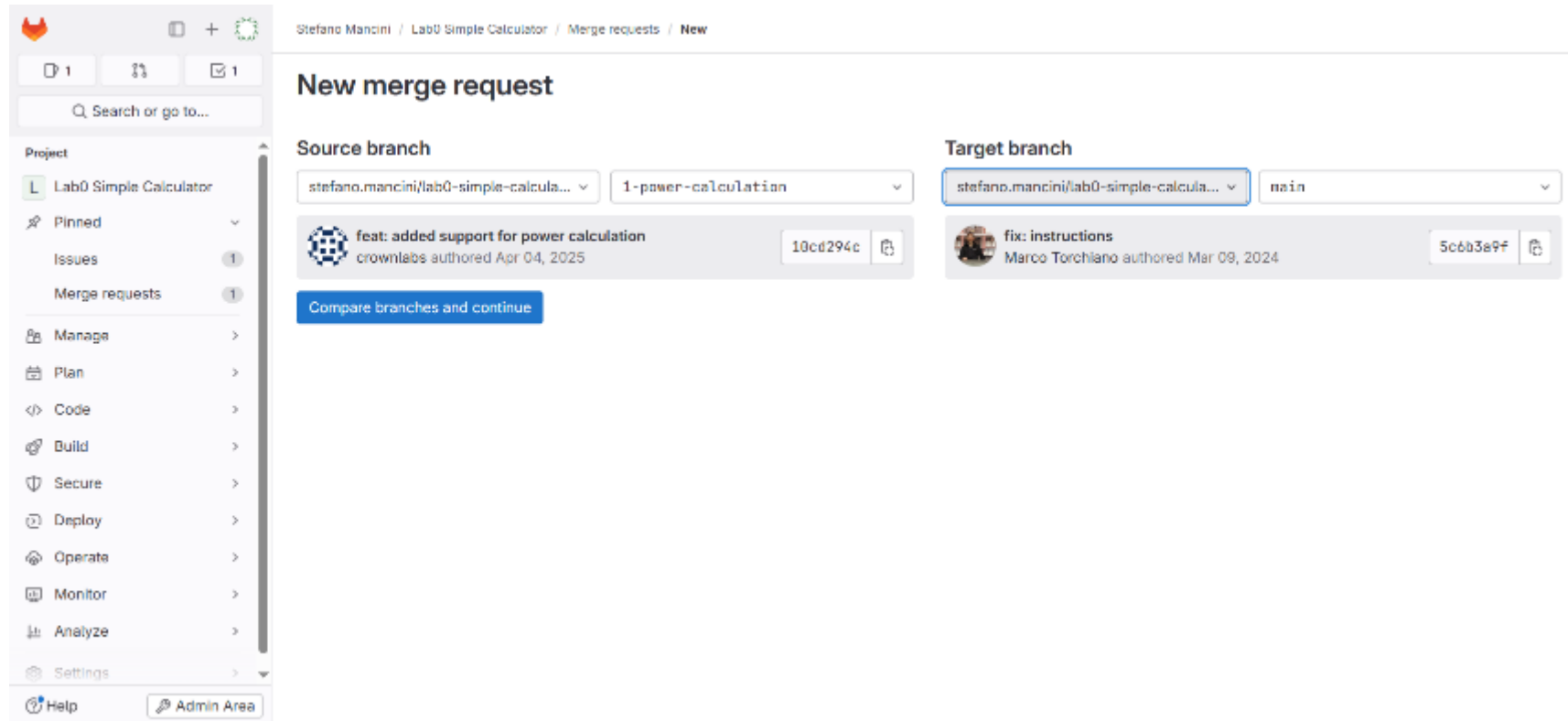
# Merge Request - features

- Propose code changes for review
- Compare two branches and see what's changed
- Discuss implementation choices with teammates
- Track the status of work: draft, open, approved, merged
- Automatically close related issues when merged

# Creating a Merge Request

- A Merge Request can be created from an existing issue
- It can also be created manually from the GitLab interface, still targeting an issue
  - Adding “Closes #7” in the description allows the automatic targeting and closing of the issue 7
- The source and target branches must be selected
- A title and description should be provided to explain the changes
- Reviewers and labels can be added to organize the review process

# Creating a Merge Request



The screenshot displays the 'New merge request' page in the GitLab web interface. On the left is a sidebar with navigation links: Project, Pinned, Issues (1), Merge requests (1), Manage, Plan, Code, Build, Secure, Deploy, Operate, Monitor, Analyze, Settings, Help, and Admin Area. The main content area is titled 'New merge request' and shows the breadcrumb 'Stefano Mancini / Lab0 Simple Calculator / Merge requests / New'. Below the title, there are two sections: 'Source branch' and 'Target branch'. The 'Source branch' section has a dropdown menu set to 'stefano.mancini/lab0-simple-calcula...' and a text input field containing '1-power-calculation'. Below this, a commit preview shows a green circle icon, the message 'feat: added support for power calculation', the author 'crownlabs', the date 'authored Apr 04, 2025', and a commit hash '10cd294c'. The 'Target branch' section has a dropdown menu set to 'stefano.mancini/lab0-simple-calcula...' and a text input field containing 'main'. Below this, a commit preview shows a green circle icon, the message 'fix: instructions', the author 'Marco Torchiano', the date 'authored Mar 09, 2024', and a commit hash '5c6b3e9f'. At the bottom of the form is a blue button labeled 'Compare branches and continue'.

Stefano Mancini / Lab0 Simple Calculator / Merge requests / New

## New merge request

Source branch

stefano.mancini/lab0-simple-calcula... 1-power-calculation

feat: added support for power calculation  
crownlabs authored Apr 04, 2025 10cd294c

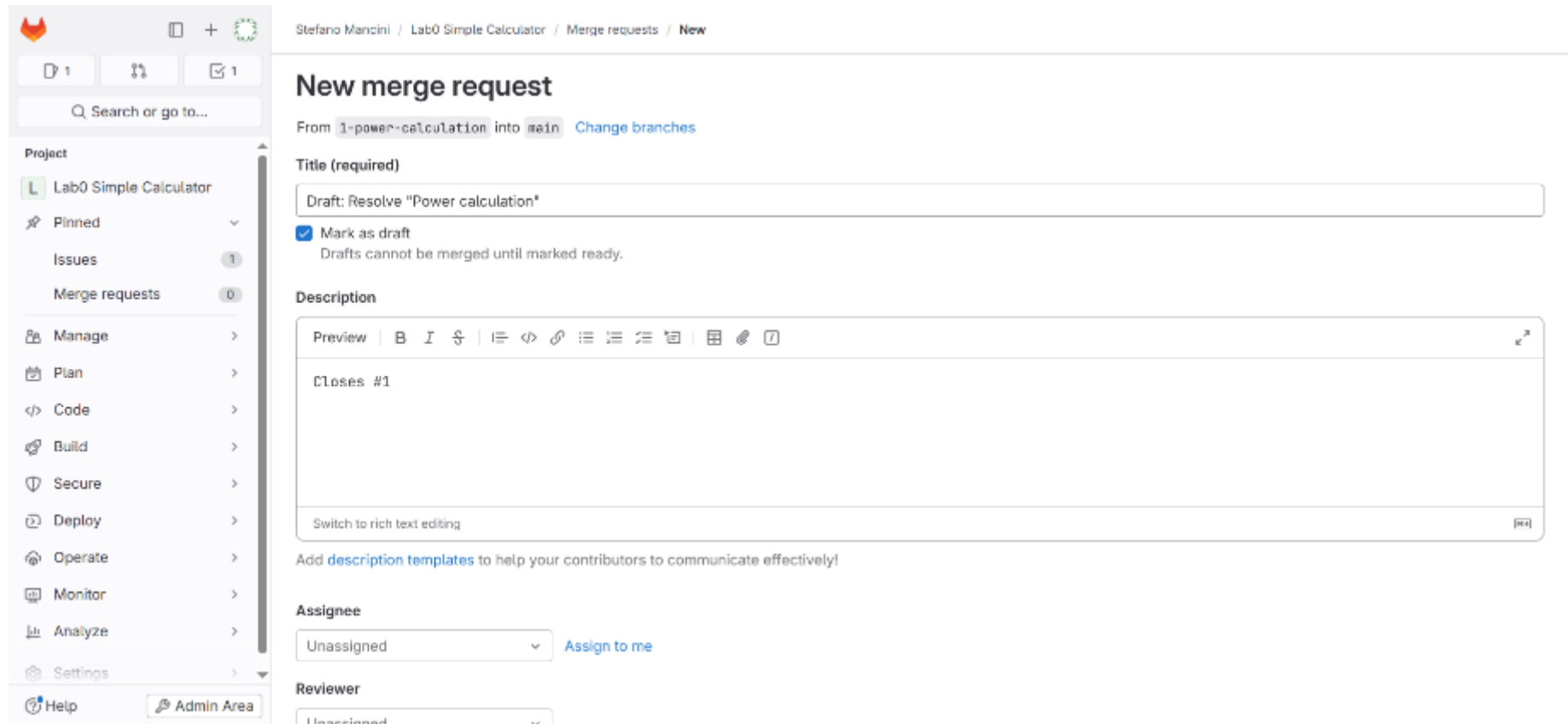
Target branch

stefano.mancini/lab0-simple-calcula... main

fix: instructions  
Marco Torchiano authored Mar 09, 2024 5c6b3e9f

Compare branches and continue

# Creating a Merge Request



The screenshot shows the 'New merge request' interface in GitLab. On the left is a sidebar with navigation links: Project, Lab0 Simple Calculator, Pinned, Issues (1), Merge requests (0), Manage, Plan, Code, Build, Secure, Deploy, Operate, Monitor, Analyze, Settings, Help, and Admin Area. The main content area is titled 'New merge request' and shows the source '1-power-calculation' being merged into the 'main' branch. The title field contains 'Draft: Resolve "Power calculation"'. The 'Mark as draft' checkbox is checked, with a note that drafts cannot be merged until marked ready. The description field contains 'Closes #1'. The 'Assignee' dropdown is set to 'Unassigned', and the 'Reviewer' dropdown is also set to 'Unassigned'.

Stefano Mancini / Lab0 Simple Calculator / Merge requests / New

## New merge request

From `1-power-calculation` into `main` [Change branches](#)

**Title (required)**

☒ Mark as draft  
Drafts cannot be merged until marked ready.

**Description**

Add [description templates](#) to help your contributors to communicate effectively!

**Assignee**

[Assign to me](#)

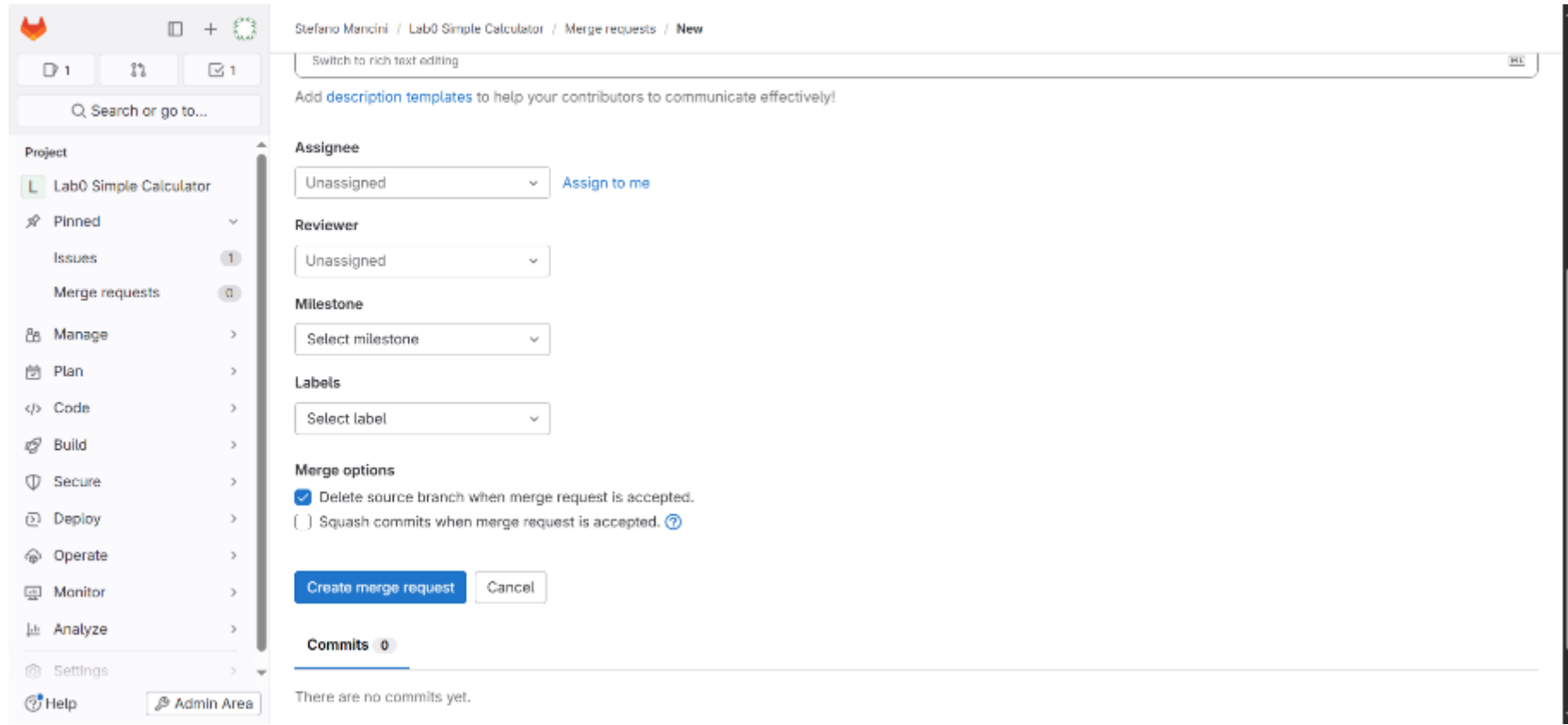
**Reviewer**

# Merge Request - configuration

- An assignee can be selected to indicate who is responsible for the work
- One or more reviewers can be added to request feedback
- A milestone can be assigned to link the MR to a broader objective
- Labels can be used to categorize or prioritize the MR
- The MR can be set to automatically delete the source branch after merging
- A squash option can be enabled to combine all commits into a single one when merging
- The MR can be marked as draft to indicate it is not ready for review yet



# Merge Request - configuration



The screenshot shows a web application interface for creating a new merge request. The left sidebar contains a navigation menu with the following items: Project, Lab0 Simple Calculator, Pinned, Issues (1), Merge requests (0), Manage, Plan, Code, Build, Secure, Deploy, Operate, Monitor, Analyze, Settings, Help, and Admin Area. The main content area is titled 'Stefano Mancini / Lab0 Simple Calculator / Merge requests / New'. It includes a 'Switch to rich text editing' button and a note about adding description templates. The configuration section includes dropdown menus for Assignee (Unassigned), Reviewer (Unassigned), and Milestone (Select milestone), along with a 'Select label' dropdown. The 'Merge options' section has two checkboxes: 'Delete source branch when merge request is accepted.' (checked) and 'Squash commits when merge request is accepted.' (unchecked). At the bottom, there are 'Create merge request' and 'Cancel' buttons, and a 'Commits 0' section indicating 'There are no commits yet.'

Stefano Mancini / Lab0 Simple Calculator / Merge requests / New

Switch to rich text editing

Add [description templates](#) to help your contributors to communicate effectively!

**Assignee**

Unassigned [Assign to me](#)

**Reviewer**

Unassigned

**Milestone**

Select milestone

**Labels**

Select label

**Merge options**

☒ Delete source branch when merge request is accepted.

☐ Squash commits when merge request is accepted. [?](#)

[Create merge request](#) [Cancel](#)

**Commits 0**

There are no commits yet.

# Code changes and feedbacks

- The Merge Request interface shows the differences between the two branches
- Code changes can be reviewed line by line
- Inline comments can be added to specific lines of code
- Discussion threads can be started to ask for clarification or suggest improvements
- This enables a structured and collaborative code review process

# Code merging

- When an MR is finally approved and gets merged into the destination branch
  - The changes from the source branch are integrated into the target branch
  - If enabled, the source branch is automatically deleted to keep the repository clean
  - If selected, all commits are squashed into a single commit before merging
  - The Merge Request status is updated to “Merged”
  - If the MR references an Issue (e.g., “Closes #12”), the Issue is automatically closed
- The code in the target branch now includes the approved changes and is ready for further development or release

# Code merging

The screenshot displays a GitHub merge request page for the repository 'Lab0 Simple Calculator'. The title of the merge request is 'Resolve "Power calculation"'. It was created by Stefano Mancini, who requested to merge the branch '1-power-calculation' into the 'main' branch, 9 minutes ago. The interface includes a left sidebar with navigation options like Pinned, Issues, Merge requests, Manage, Plan, Code, and Repository. The main content area shows the 'Overview' tab with 4 items, 1 commit, 0 pipelines, and 1 change. It indicates that the merge is 'Ready to merge!' and provides options to 'Delete source branch', 'Squash commits', or 'Edit commit message'. A 'Merge' button is prominently displayed. Below the merge options, an 'Activity' section shows a recent commit by Stefano Mancini with the message '10cd294c - feat: added support for power calculation'. On the right side, there are sections for 'Assignees', 'Reviewers', 'Labels', 'Milestone', 'Time tracking', and 'Participant', all currently showing 'None' or '1 Participant'. The bottom of the page features a footer with the 'SoftEng' logo and a Creative Commons license.

Stefano Mancini / Lab0 Simple Calculator / Merge requests / 11

## Resolve "Power calculation"

Open Stefano Mancini requested to merge 1-power-calculation into main 9 minutes ago

Overview 4 Commits 1 Pipelines 0 Changes 1

Closes #1

8 Revoke approval Approved by you

**Ready to merge!**

☒ Delete source branch ☐ Squash commits ☐ Edit commit message

1 commit and 1 merge commit will be added to main. · Closes issue #1

Merge

### Activity

Stefano Mancini added 1 commit just now

- 10cd294c - feat: added support for power calculation

[Compare with previous version](#)

Stefano Mancini @stefano.mancini started a thread on the diff just now

0 Assignees None - assign yourself Edit

0 Reviewers None - assign yourself Edit

Labels None Edit

Milestone None Edit

Time tracking No estimate or time spent

1 Participant

# Code merging

Stefano Mancini / Lab0 Simple Calculator / Merge requests / !1

## Resolve "Power calculation"

**Merged** Stefano Mancini requested to merge `1-power-calculation` into `main` 9 minutes ago

**Overview** 4 | **Commits** 1 | **Pipelines** 0 | **Changes** 1

1 unresolved thread ^ v ⋮ Add a to do

Closes #1

👍 0 🗑️ 0 😊

8✓ Approved by you 🌱

**Merged by** 🌱 Stefano Mancini just now [Revert](#) [Cherry-pick](#)

**Merge details**

- Changes merged into main with [ad054407](#).
- Did not delete the source branch.
- Closed [#1](#)

### Activity

All activity v ⌵

- Stefano Mancini** added 1 commit just now
  - [10cd294c](#) - feat: added support for power calculation[Compare with previous version](#)

**Stefano Mancini** @stefano.mancini started a thread on the diff just now ^ Hide thread

**0 Assignees** Edit  
None - assign yourself

**0 Reviewers** Edit  
None - assign yourself

**Labels** Edit  
None

**Milestone** Edit  
None

**Time tracking** ⌚ +  
No estimate or time spent

**1 Participant**  
🌱

# Code reviews

# What's a Code Review

- A Code Review is the process of examining someone else's code before it is merged into the main project.
- It is an essential step in collaborative development that helps ensure code quality, correctness, and consistency.
- Code Reviews are performed through Merge Requests, where team members can read the changes, leave comments, and suggest improvements.
- This process improves not only the code, but also helps developers learn from each other, share knowledge, and maintain a clean, understandable codebase.

# Goals

- Help catch bugs before code is merged
- Improve code readability and maintainability
- Ensure adherence to coding standards
- Promote knowledge sharing within the team
- Encourage communication and constructive feedback



# Rules

- A team should define clear coding standards and conventions
  - naming, formatting, file structure
- Review policies help decide what must be checked in every review
  - tests, documentation, error handling
- Checklists can be created to ensure consistency and completeness
  - Does it include tests?
  - Are all public methods documented?
- These rules help reviewers know what to focus on
- They also help authors write code that meets expectations from the start

# Roles

- Author: the person who wrote the code and opened the Merge Request
- Reviewer: the person who reviews the code and provides feedback
- Both roles are essential to ensure quality and shared responsibility
- Communication between author and reviewer should be respectful and focused on the code



# Etiquette

- Code reviews are a form of professional communication
- Respectful behavior helps avoid conflicts and misunderstandings
- Written feedback can be easily misinterpreted without proper tone
- Reviews should improve the code, not judge the person
- A good environment encourages learning, trust and collaboration
- Etiquette makes the process more effective and enjoyable for everyone

# Etiquette - Author

- Be open to feedback and don't take it personally
- Clarify your choices respectfully if questioned
- Fix issues quickly and explain your changes when replying
- Resolve discussions only when feedback has been addressed
- Thank reviewers for their time and input

# Etiquette - Reviewer

- Focus on the code, not the person
- Be respectful and constructive in your comments
- Ask questions instead of making demands
  - “Why not use X here?” instead of “This is wrong”
- Suggest improvements clearly and politely
- Avoid minor comments that don’t affect clarity or correctness
- Approve only when the code meets the agreed standards

# GitLab – Reviewing code

- GitLab offers two main ways to review code in a Merge Request:
  - Adding inline comments and discussions
  - Submitting a structured review with approval
- Inline comments are simple and flexible, and can be used at any time
- A formal review groups feedback and provides a clear approval status
- Submitting a review is optional:
  - A Merge Request can still be merged even without a formal review
  - This feature is useful for teams that want a more structured review process

# GitLab – Inline comments

- Inline comments can be added directly on specific lines in the “Changes” tab
- Comments are useful for asking questions, suggesting improvements, or clarifying choices
- Each comment can be marked as a discussion, allowing replies and resolutions
- Discussions stay open until explicitly resolved
- Comments can be posted immediately, without starting a full review

# GitLab – Inline Comments

The screenshot displays the GitLab web interface for a merge request. On the left, a sidebar shows the repository structure with 'Merge requests' selected. The main area shows a merge request titled 'Draft: Resolve "Power calculation"' from '1-power-calculation' into 'main'. The 'Changes' tab is active, showing a diff for 'src/calc/Calculator.java'. The diff highlights changes on lines 95, 96, and 97. An inline comment is being added on line 97, which reads: 'It would be better to perform some check on the input'. The comment is in a text box with a rich text editor toolbar. Below the comment box are buttons for 'Start a review', 'Add comment now', and 'Cancel'. The bottom of the diff shows lines 93, 94, and 95.

Stefano Mancini / Lab0 Simple Calculator / Merge requests / !1

Open Draft: Resolve "Power calculation" 1-power-calculation into main

Overview 0 Commits 1 Pipelines 0 Changes 1 Add a to do

src/calc/Calculator.java +6 -1 Viewed

```
91 93 }
92 94
95 + break;
96 + case POW:
97 + res = Math.pow(num1, num2);
```

Commenting on lines +97 to +97

Preview It would be better to perform some check on the input

Switch to rich text editing

Start a review Add comment now Cancel

```
93 98 break;
94 99 default:
95 100 error = true;
```



# GitLab – Code Review

- GitLab allows reviewers to submit a formal review after examining the code
- The review can include multiple comments grouped together
- At the end of the review, the reviewer can choose to:
  - Approve the merge request
  - Request changes with feedback
  - Leave comments without approval
- A full review provides a clear review status for the author and the team

1

1

Search or go to...

Lab0 Simple Calculator

Pinned

Issues1

Merge requests1

Manage

Plan

Code

Merge requests1

Repository

Branches

Commits

Tags

Repository graph

Compare revisions

Snippets

Stefano Mancini / Lab0 Simple Calculator / Merge requests / 1

Open

Draft: Resolve "Power calculation"

1-power-calculation

into main

Overview0

Commits1

Pipelines0

Changes1

Add a to do

src/calc/Calculator.java

+6 -1

Viewed

35

-

out.print("Enter your choice (0-4): ");

37

+

out.print("Enter your choice (0-5): ");

36

38

choice = scanner.nextI

37

39

38

40

// Check for exit cond

↓

↑

@@ -90,6 +92,9 @@ public class Calc

90

92

res = num1 / n

91

93

}

92

94

95

+

break;

96

+

case POW:

97

+

res = Math.pow(num

Stefano Mancini

@stefano.mancini

Pending

It would be better to perform some check on the in

93

98

break;

94

99

default:

95

100

error = true;

↓

Summary comment (optional)

Preview

B

I

↶

≡

</>

🔗

☰

☰

☰

📄

📎

📄

💬

No actual need

Switch to rich text editing

☐ Comment

Submit general feedback without explicit approval.

☒ Approve

Submit feedback and approve these changes.

☐ Request changes

Submit feedback that should be addressed before merging.

Submit review

Pending comments1

Finish review